

DSA : Assignment 1

1. Implement Merge Sort

Write a program to implement **Merge Sort** on an unsorted array of integers. Use the divide and conquer strategy to recursively split the array and merge it back in sorted order.

Input: [5, 2, 4, 1, 3]

Output: [1, 2, 3, 4, 5]

2. Implement Quick Sort

Write a function to implement **Quick Sort** using recursion and the partition technique (Lomuto/Hoare).

Input: [10, 7, 8, 9, 1, 5]

Output: [1, 5, 7, 8, 9, 10]

3. Find Square Root Using Binary Search

Given a non-negative integer x, return the floor of the square root using binary search (no built-in methods).

Input: x = 25

Output: 5

4. Validate Parentheses Using Stack

Check if a given string contains **valid pairs** of brackets. Use a stack to match opening and closing brackets correctly.

Input: "([{}])"

Output: true

5. Implement Stack Using Queues

Design a class MyStack that simulates stack operations using **two queues**. Implement push, pop, and top methods.

6. Implement Queue Using Stacks

Design a class MyQueue that simulates a **queue using two stacks**. Implement enqueue, dequeue, and peek operations.

7. Design Circular Queue

Design a **circular queue** data structure with fixed size. Implement methods: enqueue(), dequeue(), front(), rear(), isFull(), isEmpty().

8. Sort Colors (Dutch National Flag Problem)

Given an array with only 0s, 1s, and 2s, sort them in-place without using any library sort function.

Input: [2, 0, 2, 1, 1, 0]

Output: [0, 0, 1, 1, 2, 2]

9. Search in Rotated Sorted Array

Given a rotated sorted array, write a function to search for a given target using modified binary search.

Input: [4, 5, 6, 7, 0, 1, 2], target = 0

Output: 4

10. First and Last Occurrence of Target

Write a program to return the **first and last index** of a target value using binary search.

Input: [5, 7, 7, 8, 8, 10], target = 8

Output: [3, 4]

11. Next Greater Element Using Stack

For every element in an array, find the **next greater element to the right** using a stack.

Input: [4, 5, 2, 25]

Output: [5, 25, 25, -1]

12. Search Element in a Sorted 2D Matrix

Given a matrix where each row and column is sorted, check if a target exists using an efficient approach.

Input: [[1, 4, 7], [2, 5, 8], [3, 6, 9]], target = 5

Output: true

13. Find Kth Largest Element in Array

Write a function to return the **kth largest** element from an unsorted array. Use sorting or a heap.

Input: [3, 2, 1, 5, 6, 4], k = 2

Output: 5

14. Design Min Stack

Create a class MinStack that supports push(), pop(), top(), and retrieving the **minimum element** in O(1) time.

15. Interleave First and Second Halves of Queue

Given a queue with an even number of integers, rearrange it so that the first half interleaves with the second half.

Input: [1, 2, 3, 4, 5, 6]

Output: [1, 4, 2, 5, 3, 6]

16. Find a Peak Element in Array

An element is a **peak** if it is not smaller than its neighbors. Find any peak using binary search.

Input: [1, 3, 20, 4, 1, 0]

Output: 20

17. Sliding Window Maximum

Given an array and an integer k, return the **maximum in each sliding window** of size k.

Input: [1,3,-1,-3,5,3,6,7], k = 3

Output: [3, 3, 5, 5, 6, 7]

18. Largest Rectangle in Histogram Using Stack

Given an array of bar heights, find the area of the **largest rectangle** in the histogram.

Input: [2,1,5,6,2,3]

Output: 10

19. First Non-Repeating Character in Stream

Given a stream of characters, print the first **non-repeating character** at each insertion. Use queue + frequency array.

Input: "aabc"

Output: a a # b

20. Count Inversions in Array Using Merge Sort

Count the total number of **inversion pairs** in an array where $\text{arr}[i] > \text{arr}[j]$ and $i < j$.

Input: [2, 4, 1, 3, 5]

Output: 3 (pairs: (2,1), (4,1), (4,3))